

Adversarial Attacks and Defenses on Graph-aware Large Language Models (LLMs)

Anonymous submission

Abstract

Large Language Models (LLMs) are increasingly used as components in graph learning pipelines, extending node classification and related graph tasks beyond settings historically dominated by Graph Neural Networks (GNNs). These graph-aware systems combine relational structure with textual node information, but their robustness to adversarial manipulation is still not well characterized across model interfaces, attack timings, and input modalities. We examine this robustness question for frozen-LLM projector architectures by applying established graph adversarial attacks, including poisoning (training-time) and evasion (test-time) attacks, to two representative models, LLAGA (Chen et al. 2024) and GRAPH PROMPTER (Liu et al. 2024). We further identify a template-level attack surface in LLAGA, where malicious nodes can be inserted as placeholders in the node sequence template and substantially degrade performance. Across the evaluated benchmark datasets and experimental conditions, the results indicate that graph encoding choices can materially affect attack behavior: (1) the node sequence template in LLAGA amplifies vulnerability; (2) the GNN encoder used in GRAPH PROMPTER offers comparatively stronger robustness; and (3) both design families remain vulnerable to imperceptible feature perturbation attacks. Finally, we propose an end-to-end defense framework, GALGUARD, which combines an LLM-based feature correction module for feature-level perturbations with adapted GNN defenses for structural attacks¹

1 Introduction

Graphs represent entities and relationships through nodes and edges, providing a flexible abstraction for structured information in domains such as social networks (Fan et al. 2019), biology (Dong, Mücke, and Khosla 2022), finance (Wang et al. 2021), and healthcare (Ahmedt-Aristizabal et al. 2021). Graph Neural Networks (GNNs) (Hamilton, Ying, and Leskovec 2017; Kipf and Welling 2022; Veličković et al. 2018) were designed for this broad class of data and are now standard tools for tasks including node classification and link prediction. More recently, the general capabilities of large language models (LLMs) such as GPT-4 (Achiam et al. 2023), Gemini (Team et al. 2023), LLaMA (Touvron et al. 2023; Dubey et al. 2024), Vicuna (Chiang et al. 2023), and PaLM-2 (Anil et al. 2023) have encouraged a wider effort to couple language models with graph-structured inputs.

This family of systems, often called *graph-aware LLMs* (Xie et al. 2023), broadens the model classes available for graph tasks that were previously addressed mainly by GNNs (Ye et al. 2023; Fatemi, Halcrow, and Perozzi 2023; Kuang et al. 2023; Zhang et al. 2023; Chen et al. 2023b, 2024). Prior work further shows that graph-aware LLMs can outperform GNNs on several graph-related benchmarks (Wenkel, Wolf, and Knyazev 2023; Chen et al. 2023b; Zhu et al. 2024; Chen et al. 2024), making them practical alternatives in some experimental regimes.

As graph-aware LLMs become plausible components in graph learning workflows, adversarial robustness becomes a cross-cutting design concern rather than a model-specific edge case. Existing attacks on GNNs, including poisoning (training-time attacks) and evasion (test-time attacks) (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019; Dai et al. 2023), commonly assume a black-box adversary without access to the model architecture or parameters. Such attacks compromise *model integrity* by subtly changing graph structure or node features to reduce predictive performance. Whereas the GNN robustness literature has largely emphasized structural perturbations, graph-aware LLMs also expose textual node features to language-model processing, making visually imperceptible feature perturbations relevant to a broader set of text-attributed graph pipelines.

These observations motivate our central question: *do adversarial attacks designed for GNNs remain effective when graph tasks are performed by graph-aware LLMs?* The answer depends not only on the attack type, but also on the interface through which graph information reaches the LLM. Textual node features are naturally compatible with LLM inputs, whereas graph structure must be transformed by mechanisms that preserve relational information (Chen et al. 2024; Guo et al. 2023; Liu and Wu 2023; Zhang et al. 2023; Sun et al. 2023; Bi et al. 2024; Pan et al. 2024). To frame this design space, we categorize graph-aware methods into two broad classes: approaches that express graphs as *textual descriptions*, and approaches that use *learned projectors* to bridge graph and text representations. Because conventional text attacks are already expected to affect LLMs that consume graph descriptions as plain text, we focus on learned projector methods and evaluate two computationally efficient representatives, LLAGA (Chen et al. 2024) and GRAPH-

¹Our code will be released upon acceptance.

PROMPTER (Liu et al. 2024), both of which use frozen pre-trained LLMs.

Our results show that graph encoding decisions can affect robustness across attacks, datasets, and evaluation stages, suggesting that robustness should be analyzed at the graph-to-LLM interface rather than only at the foundation-model layer. In particular, we identify an attack surface in LLAGA in which an adversary can inject malicious nodes as placeholders within the node sequence template and substantially degrade performance. We also propose an end-to-end graph-aware LLM defense, GALGUARD, which combines an LLM-based feature corrector with adapted GNN-based structural defenses. To our knowledge, this is the first systematic study of adversarial vulnerability in graph-aware LLMs. The contributions of the paper are summarized as follows:

- We use representative GNN adversarial attacks, specifically NETTACK (Zügner, Akbarnejad, and Günnemann 2018) and METAATTACK (Zügner and Günnemann 2019), to evaluate how poisoning and evasion attacks carry over to graph-aware LLMs.
- We organize graph-aware LLM methods by their graph encoding approaches, clarifying how different interface designs incorporate graph structure into LLM-based pipelines.
- We identify a node sequence template attack surface in LLAGA, showing that malicious placeholder nodes can degrade performance under the evaluated conditions.
- We assess graph-aware LLM vulnerabilities under imperceptible textual feature perturbations, extending robustness analysis beyond the structural attacks most often studied for GNNs.
- We propose GALGUARD, an end-to-end defense strategy that pairs LLM-based feature correction with adapted GNN-based structural defenses.

2 Graph Encoding-Based Taxonomy of LLM Adaptations

Graph encoding methods for LLMs can be grouped into two broad families: *textual descriptions of graphs*, where graph information is rendered as natural language, and *learned graph projectors*, where a trained module maps graph data into embeddings or structured inputs that an LLM can process. This taxonomy captures the principal adaptation strategies relevant to our robustness study and emphasizes that the same graph task may expose different attack surfaces depending on the graph-to-language interface. Figure 5 illustrates both families. Appendix A.1 gives a fuller description, and Appendix A.2 reviews related work on GNN attacks.

Graph Representation as a Textual Description. Textual-description approaches convert graph data into natural language so that an LLM can reason over the graph through prompts or instructions (Guo et al. 2023; Liu and Wu 2023; Ye et al. 2023; Wenkel, Wolf, and Knyazev 2023; Wei et al. 2024b; Fatemi, Halcrow, and Perozzi 2023; Tan et al. 2023; Zhao et al. 2023; Wei et al. 2024a; Zhang 2023). Related multimodal work by Liu et al. (2023) and Su et al. (2022)

connects molecule graphs with textual descriptions. In addition, Huang et al. (2023a) studied how graph structure can be incorporated into prompts, finding that LLMs often process graph prompts as contextual paragraphs, which makes prompt design central across settings such as (Brannon et al. 2023; Zhang et al. 2023; Sun et al. 2023; Bi et al. 2024; Pan et al. 2024; He et al. 2024). Other methods use LLMs to enhance node features, refine graph structures, act as predictors, or serve as feature extractors for GNNs (Chen et al. 2023a; Chandra et al. 2020; Chen et al. 2023b; Mavromatis et al. 2023). For inference, Zhu et al. (2024) proposed a fine-tuning method for textual graphs that combines LLMs and GNNs, and Duan et al. (2023) introduced an approach for improving textual graph learning. These methods offer a direct route for applying LLMs to graph tasks across domains, but they may struggle to preserve structural detail, especially for large graphs, and can depend strongly on prompt engineering.

Graph Representation as a Learned Projector. Learned-projector methods address some limitations of textual descriptions by mapping graph data into embeddings or graph vectors that can be consumed more directly by an LLM, creating a reusable interface between graph encoders and frozen language models. This family includes LLAGA, which uses node-level templates and a linear projector (Chen et al. 2024), as well as methods that incorporate GNN layers into a pre-trained LLM (Liu et al. 2024; Qin et al. 2023). Other variants use graph neural prompts to encode knowledge graphs (Tian et al. 2024), construct prompt-based node feature extractors with a GNN adapter (Huang et al. 2023b), or compress graph information into fixed-length prefixes using graph transformers (Chai et al. 2023; Yang et al. 2021; Peng et al. 2024). This range of designs makes the projector interface a natural level at which to study robustness across graph-aware LLM systems.

This Work: We study projector methods built on *frozen pre-trained LLMs* because this design is computationally efficient and avoids the cost of retraining the language model (Chen et al. 2024; Liu et al. 2024; Qin et al. 2023; Tian et al. 2024; Huang et al. 2023b). Our experiments use two representative models from this family: LLAGA (Chen et al. 2024), which uses node-level templates with a linear projector, and GRAPH-PROMPTER (Liu et al. 2024), which combines a GNN with a linear projector. We exclude textual-description methods from the main evaluation because their scalability challenges and plain-text interface make them less aligned with the graph-specific projector vulnerabilities examined in this work, while the taxonomy still positions our findings within the broader graph-aware LLM landscape.

3 Threat Model

We define the threat model by adapting standard assumptions from adversarial graph learning to graph-aware LLMs (Zügner, Akbarnejad, and Günnemann 2018; Wu et al. 2019; Zügner and Günnemann 2019; Ma, Ding, and Mei 2020; Jin et al. 2021; Dai et al. 2023; Li et al. 2023). The setting captures application scenarios in which an adversary can influence only a limited portion of a larger graph while the deployed graph-aware model must still reason over the resulting structure and text. For example, in a social network,

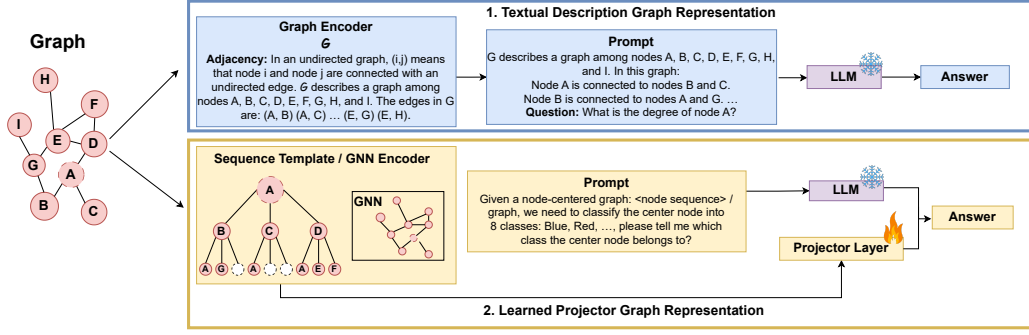


Figure 1: **Graph encoding-based adaptations of LLMs.** Textual-description methods express graph information in natural language (top), whereas learned-projector methods map graph information into LLM-consumable tokens through node-template sequences or GNN encoders (bottom). In the frozen settings shown here, the LLM remains fixed and only the graph-to-token interface is trained, illustrating the model family examined in this work.

an attacker may control a small number of bot accounts and use them to evade detection or affect the classification of legitimate users. In a document graph such as Wikipedia, sparsely connected hoax articles may be linked strategically to influence the classification of genuine pages. Such settings arise whenever graph-aware LLMs are considered for recommendation, social media analysis, healthcare, or other graph-dependent decision pipelines that may otherwise rely on GNNs. Following prior work on graph attacks, we characterize the threat model along model access, adversarial capabilities, attack strategies, and the attacker’s goal.

Model Access. The attacker operates in a black-box setting and has no knowledge of the model architecture or parameters. The attacker can influence model behavior only indirectly through training or test data, matching deployment settings where graph-aware LLM services expose predictions but not internal representations.

Adversarial Capabilities. Consistent with existing poisoning and evasion attacks, the attacker has access to the training data for poisoning or the inference data for evasion. Subject to a predefined budget and unnoticeability constraints, the attacker may modify or inject data, covering both structural graph edits and textual node-feature changes.

Attack Strategies. The attacker uses feature manipulation, by altering node features, and structure manipulation, by adding or removing edges, reflecting the dual input channels used by graph-aware LLMs.

Attacker’s Goal. The attacker aims to compromise model integrity by degrading overall performance, measured through an increased misclassification rate for the classification model.

4 Experimental Setup

Dataset. We evaluate on Cora, Citeseer, and PubMed (Yang, Cohen, and Salakhudinov 2016), three widely used text-attributed graph benchmarks that instantiate document-linked relational data for graph-aware LLM systems. Together, these datasets cover small- to medium-scale citation graphs with different sparsity levels, making them standard testbeds for the model class examined here (Chen et al. 2023a, 2024; Liu

et al. 2024). To assess whether the same qualitative behavior extends beyond these smaller citation graphs, we also include ArXiv (Hu et al. 2020) in an additional larger-scale experiment (see Appendix A.5). Following (Chen et al. 2024), we use standard train/validation/test splits of 6:2:2 for the three smaller datasets and 5:2:3 for ArXiv. Table 1 reports the dataset statistics for the main evaluation.

Table 1: Statistics for the text-attributed graph benchmarks used in the main evaluation setting.

Data	#Node	#Edge	Sparsity (%)
Cora	2708	5429	14.81
CiteSeer	3327	4614	8.34
PubMed	19717	44338	2.28

Tasks. We use *node classification* as the primary task because it is a common graph machine learning benchmark and a standard target for graph adversarial attacks (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019; Liu and Wu 2023). This task captures a broader prediction setting in which graph-aware LLMs must combine node text with relational context.

Evaluation Metrics. We report *accuracy*, the proportion of correctly predicted labels, to align with both graph-learning and graph-attack evaluation practice. Each experiment is repeated three times, and all tables report the mean and standard deviation so that robustness trends can be compared across datasets, attack timings, and model designs. All experiments were conducted on a machine equipped with an NVIDIA A40 GPU with 48GB of memory.

Choice of Graph-aware LLM and Adversarial Attack Methods. We next specify the graph-aware LLM architectures and adversarial attack methods used to evaluate robustness within the frozen-LLM projector setting considered in this work.

Graph-Aware LLM Methods. We use two representative frozen-LLM projector methods: LLAGA (Chen et al. 2024) and GRAPHPROMPTER (Liu et al. 2024). Both keep the

Table 2: Performance of representative poisoning and evasion graph attacks, NETTACK and METAATTACK, on the graph-aware LLMs LLAGA and GRAPHPROMPTER across Cora, Citeseer, and PubMed.

Attack	Type	Cora		Citeseer		PubMed	
		LLAGA	GRAPHPROMPTER	LLAGA	GRAPHPROMPTER	LLAGA	GRAPHPROMPTER
Clean	—	0.89 $\pm$ 0.07	0.60 $\pm$ 0.03	0.71 $\pm$ 0.04	0.70 $\pm$ 0.03	0.90 $\pm$ 0.04	0.90 $\pm$ 0.02
NETTACK	Poisoning	0.87 $\pm$ 0.04 (2%)	0.60 $\pm$ 0.03 (0%)	0.64 $\pm$ 0.05 (10%)	0.68 $\pm$ 0.02 (3%)	0.89 $\pm$ 0.03 (1%)	0.88 $\pm$ 0.02 (2%)
	Evasion	0.55 $\pm$ 0.09 (38%)	0.53 $\pm$ 0.05 (12%)	0.59 $\pm$ 0.04 (19%)	0.61 $\pm$ 0.03 (14%)	0.84 $\pm$ 0.05 (7%)	0.81 $\pm$ 0.02 (10%)
METAATTACK	Poisoning	0.79 $\pm$ 0.03 (11%)	0.58 $\pm$ 0.05 (3%)	0.63 $\pm$ 0.04 (12%)	0.65 $\pm$ 0.06 (7%)	0.89 $\pm$ 0.08 (1%)	0.85 $\pm$ 0.02 (6%)
	Evasion	0.44 $\pm$ 0.06 (51%)	0.52 $\pm$ 0.04 (13%)	0.50 $\pm$ 0.03 (35%)	0.56 $\pm$ 0.02 (22%)	0.73 $\pm$ 0.04 (19%)	0.77 $\pm$ 0.03 (14%)

pre-trained LLM frozen, a computationally efficient design that can be reused across graph tasks and LLM backbones without retraining the language model. LLAGA represents template-based graph encoding through node-level sequences and a linear projector, whereas GRAPHPROMPTER represents GNN-encoder-based graph encoding followed by a linear projector. In both models, the projector is a 2-layer multi-layer perceptron (MLP). The foundation LLMs for LLAGA and GRAPHPROMPTER are Vicuna-7B (Chiang et al. 2023) and LLAMA2-7B (Touvron et al. 2023), respectively.

Adversarial Attack Methods. We evaluate poisoning and evasion variants of two representative graph adversarial attacks, NETTACK (Zügner, Akbarnejad, and Günnemann 2018) and METAATTACK (Zügner and Günnemann 2019), to test whether attack mechanisms developed for GNNs also apply to graph-aware LLM pipelines. NETTACK is a targeted attack that searches for minimal perturbations causing specific node misclassification, while METAATTACK is an untargeted attack that optimizes graph structure as a hyperparameter. For NETTACK, we follow (Li et al. 2023; Zügner and Günnemann 2019) by targeting each node sequentially and aggregating the outcomes, yielding an untargeted evaluation protocol. Across experiments, we perturb only 10% of the edges to preserve the unnoticeability constraint used in (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019). Appendix A.2 provides full details for METAATTACK and the related attack setup.

5 Results

5.1 Poisoning and Evasion Attacks on Graph-aware LLMs

We study poisoning and evasion attacks by adapting NETTACK (Zügner and Günnemann 2019) and METAATTACK (Zügner and Günnemann 2019) to graph-aware LLM pipelines that consume both relational structure and textual node attributes. For poisoning attacks on LLAGA, the perturbed graph is used to generate node-level sequences before retraining the projector. For GRAPHPROMPTER, the perturbed graph is re-encoded with a GAT before retraining the projector. For evasion attacks, both models are trained on clean data and graph perturbations are introduced only at inference time, matching deployment scenarios in which a trained graph-aware model encounters manipulated graph inputs.

Results. Table 2 shows three main patterns across the evaluated graph-aware LLM settings. First, poisoning attacks have limited impact, with a maximum performance degradation of 11% for both attack methods, suggesting that learned representations and additional LLM context can provide some resilience when perturbations are present during training. Evasion attacks are more damaging, reaching performance degradation of up to 51%, which indicates that test-time graph manipulation remains relevant for frozen-projector deployments. Second, METAATTACK generally outperforms NETTACK for both poisoning and evasion across datasets, consistent with its ability to optimize small but effective edge perturbations by treating graph edges as hyperparameters. Third, denser datasets appear more vulnerable than sparser ones under these attacks; for example, LLAGA drops by 51% on the denser Cora graph under evasion, compared with 19% on the sparser PubMed graph.

5.2 Which Graph-aware Paradigm is More Susceptible to Graph Adversarial Attacks and Why?

The results in Table 2 indicate that LLAGA is more susceptible to graph adversarial attacks than GRAPHPROMPTER because its encoding pipeline is more *sensitive to structural perturbations*. LLAGA incorporates graph structure by constructing a fixed-length node-level sequence for each node and by computing a graph Laplacian embedding. Each sequence position corresponds to a relative structural location in the original graph, so structural perturbations can alter the information passed to the LLM through multiple parts of the template. By contrast, GRAPHPROMPTER’s textual node embeddings remain unchanged under structural perturbations because only the GNN component is directly affected. This separation allows clean textual features to compensate for perturbed graph structure, suggesting a robustness advantage for architectures that decouple textual and structural channels under the evaluated conditions.

6 Node Sequence Template Injection Attacks

We next examine a graph-aware LLM vulnerability that arises from the node sequence construction used by LLAGA and is therefore informative for template-based graph encoders more generally. The attack is illustrated in Figure 2, and the full algorithm is given in Appendix A.4. To obtain the

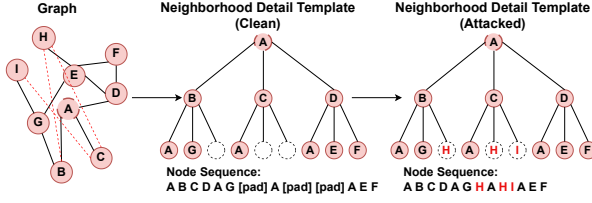


Figure 2: Illustration of node sequence template injection attacks in a sampled-neighborhood graph encoding. The example shows Non-adjacent Injection (NI), where red dashed lines denote attacker-added edge perturbations. For Supernode Injection (SI) and Multiple Supernode Injection (MSI), one or more supernodes are inserted into placeholder positions. Here, $k = 3$.

node sequence for a node u , the neighborhood detail template in LLAGA constructs a *fixed-shape* computational tree whose children are sampled 1-hop neighbors. This sampling is repeated for later tree levels. When a node has too few neighbors, placeholders are inserted to preserve the fixed size, e.g., $k = 10$ children, following LLAGA’s original design (see “Clean” from Figure 2 and lines 1-14 in the algorithm). This fixed-shape requirement creates an attack surface for graph-aware LLM designs that represent neighborhoods through sampled sequences.

The attack manipulates the graph so that malicious or uninformative nodes occupy placeholder positions in LLAGA’s neighborhood detail template, thereby degrading the representation supplied to the projector. We consider three strategies that cover increasingly influential forms of placeholder control. *Non-Adjacent Injection (NI)* injects nodes that are outside the 2-hop neighborhood of a target node u (see “Attacked” from Figure 2 and lines 21-27). *Supernode Injection (SI)* inserts a high-degree node as a placeholder while keeping it outside the target node’s 2-hop neighborhood. *Multiple Supernode Injection (MSI)* extends *SI* by injecting multiple distinct supernodes as placeholders (lines 35-43). In all three cases, the injected nodes corrupt the target node sequence, illustrating how sequence construction can become a robustness bottleneck for template-based graph-aware LLMs.

Results. As shown in Table 3, LLAGA remains relatively resilient to poisoning attacks on Cora and Citeseer, but its accuracy drops substantially under evasion. On Cora, accuracy decreases to 0.42, 0.36, and 0.30 under *NI*, *SI*, and *MSI*, respectively; on Citeseer, the corresponding accuracies are 0.36, 0.30, and 0.24. These results indicate that template manipulation is most consequential when the corrupted sequence is encountered at inference time rather than absorbed during projector training.

On the larger PubMed dataset, the same injection strategies have a smaller effect, with evasion accuracies of 0.85, 0.82, and 0.76. A plausible explanation is that localized injections in smaller and more tightly connected graphs such as Cora and Citeseer have a more concentrated effect on the local neighborhood representation. In the larger PubMed graph, the same intervention is more diluted, showing that the practical impact of this attack surface depends on dataset scale and

Table 3: Node sequence template injection results across datasets and injection strategies for the template-based graph-aware LLM setting. *NI*, *SI*, and *MSI* denote non-adjacent, single-supernode, and multiple-supernode injection, respectively. Clean reports unattacked model performance.

Data	Attack	NI	SI	MSI
Cora	Clean	—	0.89 ± 0.07	—
	Poisoning	0.86 ± 0.03	0.84 ± 0.06	0.82 ± 0.04
	Evasion	0.42 ± 0.08	0.36 ± 0.05	0.30 ± 0.05
Citeseer	Clean	—	0.71 ± 0.04	—
	Poisoning	0.69 ± 0.05	0.66 ± 0.03	0.62 ± 0.02
	Evasion	0.36 ± 0.06	0.30 ± 0.04	0.24 ± 0.05
PubMed	Clean	—	0.90 ± 0.04	—
	Poisoning	0.88 ± 0.04	0.85 ± 0.05	0.81 ± 0.03
	Evasion	0.85 ± 0.07	0.82 ± 0.04	0.76 ± 0.03

connectivity.

Relative to the structural graph attacks in Table 2, template injection is often more damaging in the evaluated settings. For example, *MSI* yields evasion accuracies of 0.30, 0.24 and 0.76 on Cora, CiteSeer and PubMed, respectively, whereas METAATTACK reaches 0.44, 0.50 and 0.73.

7 Poisoning and Evasion Attack via Feature Perturbation

Most graph adversarial attacks focus on structural perturbations (Li et al. 2023; Wu et al. 2019; Dai et al. 2018; Sun et al. 2022; Zügner and Günnemann 2019); NETTACK (Zügner, Akbarnejad, and Günnemann 2018) also includes feature perturbations, but its feature component does not transfer directly to textual node attributes. Because graph-aware LLMs use node text as part of the prediction pipeline, robustness evaluation should cover textual feature perturbations as well as graph topology. We therefore evaluate unnoticeable feature perturbation attacks, especially in settings where structural attacks have only moderate impact on a more robust model such as GRAPH PROMPTER.

We adapt the imperceptible attack strategies of (Boucher et al. 2022) with one modification: perturbations are generated using a task-agnostic English-to-French translation objective rather than labels from a specific graph benchmark. The attack changes input text in ways that are visually identical or nearly identical to humans but induce incorrect translation, thereby disrupting the encoded semantics available to downstream graph-aware models. The intuition is that if the translated output is wrong, then the semantic representation produced from the text is also likely to be corrupted. We use two imperceptible attacks: *Homoglyphs*, which replace characters with visually similar alternatives, and *Reorderings*, which use direction-control characters to alter rendering order. Appendix A.6 provides examples of both attacks.

Following (Boucher et al. 2022), we optimize these perturbations with differential evolution (Storn and Price 1997), maximizing the Levenshtein distance between model outputs for perturbed and unperturbed text. The perturbation budget is 10% of the average length of each textual feature, keeping

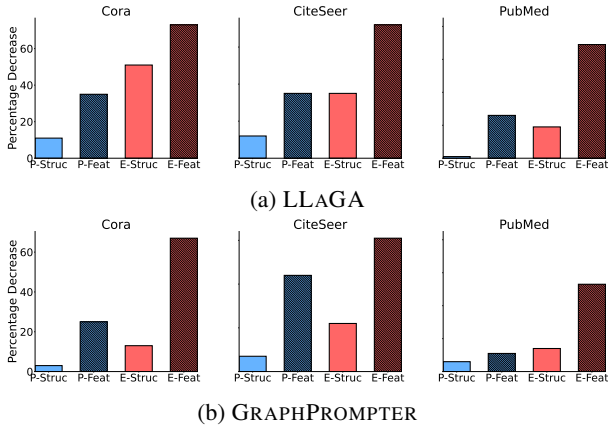


Figure 3: Comparison of structure-based and feature-based adversarial attacks on graph-aware LLMs. Attack success is measured as the percentage decrease from clean accuracy on the y-axis, showing how textual and relational attack surfaces differ under the same evaluation protocol. P-Struc, P-Feat, E-Struc, and E-Feat denote poisoning and evasion attacks based on structural or feature perturbations.

the attack condition consistent across datasets and model classes.

Results. As shown in Table 4, Reordering attacks are generally stronger than Homoglyph attacks across the evaluated models and datasets. On Cora and CiteSeer, LLAGA is vulnerable to both attack types: poisoning produces accuracy drops of 27% and 23% under Homoglyph, and larger drops of 35% and 25% under Reordering, respectively. In both cases, GRAPH PROMPTER is more resilient, but it is not immune to feature perturbations. PubMed follows the same qualitative pattern, although the overall effect is smaller than on the smaller datasets.

Comparing feature and structural perturbations in Figure 3 reveals a broader vulnerability pattern for graph-aware LLMs. Feature perturbations, especially Reordering, outperform structural perturbations from METAATTACK by a large margin on both LLAGA and GRAPH PROMPTER. This indicates that, in the evaluated frozen-projector models, textual node features can be a stronger attack surface than graph structure. The next section therefore studies a unified setting that combines both perturbation types.

8 A Unified Imperceptible Attack on Graph-aware LLMs

We further evaluate a unified attack that combines feature and structural perturbations. This setting reflects the broader attack surface of graph-aware LLMs, where relational structure and textual node attributes can both influence predictions in the same pipeline. We hypothesize that combining the two perturbation types will degrade accuracy more than either one alone while preserving inconspicuous changes to both graph structure and features.

For the structural component, we use METAATTACK, which subtly changes graph edges. For the feature component,

we combine Homoglyph and Reordering attacks as described in the previous section. Together, these perturbations target both relational and textual signals, approximating a more general adversarial condition for models that jointly consume graph topology and node text.

Results. As shown in Table 5, jointly applying structural and feature perturbations degrades performance more than either perturbation type alone across the evaluated datasets. This result highlights a robustness distinction between graph-aware LLMs and traditional GNN-centered settings: graph-aware LLMs can be affected strongly by feature-level perturbations as well as structural changes. The unified attack is effective because these models rely on consistency between graph topology and textual node attributes, making combined inconspicuous modifications especially harmful under the tested conditions.

9 Ablation Study

9.1 Varying Feature Perturbation

To characterize how attack strength scales with perturbation level, we vary the percentage of perturbed textual features. We focus on the Reordering attack, which follows trends similar to Homoglyph, and vary the perturbation percentage from 1% to 10%. Across the evaluated models and datasets (Cora, CiteSeer, and PubMed), reducing the feature perturbation rate makes the attack less successful, as expected. As the perturbation level decreases, poisoning and evasion success rates also tend to converge. With fewer perturbations, the adversarial signal is weaker and has less impact regardless of whether it is introduced during training (poisoning) or inference (evasion), suggesting that perturbation budget is a key scope condition for feature-level robustness claims. The Appendix (Figure 2) provides a detailed plot of these trends.

9.2 Does the Choice of LLM Affect Attack Performance?

We also test whether the foundation LLM backbone changes adversarial robustness within the same graph-aware architecture. LLAGA uses Vicuna-7B (Chiang et al. 2023), whereas GRAPH PROMPTER uses Llama2-7B (Touvron et al. 2023). We replace LLAGA’s backbone with Llama2-7B and compare poisoning and evasion performance across datasets. The clean accuracies are similar (e.g., Vicuna: 0.89 ± 0.07 , Llama2: 0.90 ± 0.03 on Cora), and the models show comparable attack vulnerability. The same qualitative trend holds when swapping LLMs in GRAPH PROMPTER. These results suggest that, under our evaluated conditions, the graph encoding mechanism has a larger effect on adversarial robustness than the specific foundation LLM choice, a useful boundary for generalizing from the tested backbones to related frozen-LLM projector systems. Full results appear in Table 2 of the Appendix.

10 Defense: GALGUARD

Existing GNN defenses do not directly cover the graph-aware LLM setting because these models can be vulnerable to both

Table 4: Performance under imperceptible textual feature perturbation attacks on graph-aware LLMs; parentheses report percentage drop from clean accuracy.

Data	Method	Poisoning		Evasion	
		LLAGA	GRAPHPROMPTER	LLAGA	GRAPHPROMPTER
Cora	Homoglyph	0.65 $\pm$ 0.05 (27%)	0.52 $\pm$ 0.04 (13%)	0.41 $\pm$ 0.04 (54%)	0.38 $\pm$ 0.03 (37%)
	Reordering	0.58 $\pm$ 0.06 (35%)	0.45 $\pm$ 0.05 (25%)	0.24 $\pm$ 0.05 (73%)	0.20 $\pm$ 0.03 (67%)
Citeseer	Homoglyph	0.54 $\pm$ 0.04 (23%)	0.65 $\pm$ 0.04 (7%)	0.32 $\pm$ 0.03 (55%)	0.52 $\pm$ 0.05 (26%)
	Reordering	0.46 $\pm$ 0.03 (35%)	0.39 $\pm$ 0.02 (44%)	0.20 $\pm$ 0.05 (72%)	0.27 $\pm$ 0.02 (61%)
PubMed	Homoglyph	0.72 $\pm$ 0.04 (20%)	0.83 $\pm$ 0.04 (8%)	0.36 $\pm$ 0.06 (60%)	0.55 $\pm$ 0.03 (39%)
	Reordering	0.67 $\pm$ 0.05 (26%)	0.80 $\pm$ 0.03 (11%)	0.28 $\pm$ 0.04 (69%)	0.42 $\pm$ 0.04 (53%)

Table 5: Model accuracy under the unified imperceptible feature-and-structure attack; parentheses report percentage drop from clean accuracy.

Data	Attack	LLAGA	GRAPHPROMPTER
Cora	Poisoning	0.35 $\pm$ 0.05 (61%)	0.28 $\pm$ 0.08 (53%)
	Evasion	0.14 $\pm$ 0.06 (84%)	0.11 $\pm$ 0.05 (82%)
Citeseer	Poisoning	0.26 $\pm$ 0.04 (63%)	0.22 $\pm$ 0.03 (69%)
	Evasion	0.10 $\pm$ 0.09 (86%)	0.13 $\pm$ 0.07 (81%)
PubMed	Poisoning	0.39 $\pm$ 0.07 (57%)	0.32 $\pm$ 0.04 (64%)
	Evasion	0.12 $\pm$ 0.05 (87%)	0.19 $\pm$ 0.04 (79%)

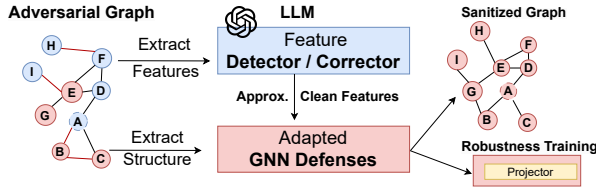


Figure 4: **Overview of GALGUARD defense.** Perturbed node features and edges are colored in blue and red, respectively. The LLM detects anomalous textual node features from an adversarial graph and generates corrected versions. The refined features, together with the perturbed edges, are then passed to GNN defenses that can sanitize graph inputs or be adapted for robust projector training in graph-aware LLM pipelines.

structural and textual feature perturbations. Many graph defenses prune or reweight edges using node feature similarity (Wu et al. 2019; Dai et al. 2018; Zhang and Zitnik 2020; Jin et al. 2020; Zhang and Ma 2024), which assumes that node features remain clean. When textual features are also perturbed, these methods can fail (see Figure 3 in the Appendix). Likewise, defenses that decouple structure and features (Shanthamallu, Thiagarajan, and Spanias 2021; Wu et al. 2021), as well as OCR-based sanitization (Boucher et al. 2022), are insufficient under coordinated feature-and-structure attacks. This combined vulnerability motivates an integrated defense for graph-aware LLM pipelines that expose both node text and graph topology to adversarial influence.

We therefore propose graph-aware LLM defense (GALGUARD), an end-to-end strategy that combines two complementary components applicable to the evaluated frozen-projector setting: (1) an LLM-based feature corrector and (2) adapted GNN-based structural defenses.

Feature Robustness Strategy. As shown in Figure 4, the LLM corrector analyzes textual node features for anomalous patterns and produces approximate corrected text. This step aims to restore feature integrity before the graph-aware model consumes the node attributes, thereby reducing the impact of feature-level perturbations across graph-aware pipelines that rely on text-attributed nodes. The instructions and prompts used for this correction module are provided in Appendix

A.6. In our experiments, GPT-4 Turbo serves as the LLM corrector.

Graph Structure Robustness Strategy. For structural robustness, we use a two-step strategy that adapts established GNN defenses to the graph-aware LLM interface. First, we apply a *graph purification heuristic* (Wu et al. 2019) as pre-processing. For each node v , we compute the cosine similarity between its embedding $\phi(x_v)$ and each neighbor embedding $\phi(x_u)$, then remove low-similarity edges. Second, we incorporate *GNNGuard* (Zhang and Zitnik 2020). For GRAPHPROMPTER, we apply GNNGuard principles directly to the message-passing layers. For LLAGA, we adapt the defense by incorporating purification into construction of the computational tree used for its node sequence. We also introduce a learnable *global structural context embedding* (M_{global}) as a parameter in LLAGA’s projector. By concatenating M_{global} with the purified node sequence embedding and jointly training them, the model can better operate on potentially perturbed graph inputs. Because GALGUARD is modular, other graph defenses, e.g. Pro-GNN (Jin et al. 2020), can replace GNNGuard.

10.1 Results

Table 6 reports model accuracy under METAATTACK and under two defense variants: GALGUARD_p, which combines feature correction with graph purification, and full GALGUARD, which adds GNNGuard or its architecture-specific adaptation. Across Cora, CiteSeer, and PubMed, and for both poisoning and evasion, the defenses mitigate the impact of METAATTACK. GALGUARD_p provides a useful baseline defense by combining LLM-based feature correction with initial graph purification. Full GALGUARD consistently improves robustness beyond GALGUARD_p, indicating that feature correction and GNN-inspired structural defense are complementary for graph-aware LLMs that jointly rely on textual and relational inputs.

11 Conclusion

This paper provides a first systematic study of adversarial attacks and defenses for graph-aware LLMs in the frozen-projector setting. The results show that the way graph information is integrated into an LLM can itself become a robustness factor. In particular, the node sequence template

Table 6: Performance under METAATTACK and the proposed GALGUARD variants for poisoning and evasion attacks in graph-aware LLMs. GALGUARD_p combines feature correction with graph purification.

Data Method		LLAGA		GRAPHPROMPTER	
		Poisoning	Evasion	Poisoning	Evasion
Cora	METAATTACK	0.79 $\pm$ 0.03	0.44 $\pm$ 0.06	0.58 $\pm$ 0.05	0.52 $\pm$ 0.04
	GALGUARD _p	0.82 $\pm$ 0.05	0.62 $\pm$ 0.04	0.59 $\pm$ 0.03	0.56 $\pm$ 0.04
	GALGUARD	0.85$\pm$0.04	0.83$\pm$0.04	0.60$\pm$0.05	0.59$\pm$0.03
CiteSeer	MetaAttack	0.63 $\pm$ 0.04	0.50 $\pm$ 0.03	0.65 $\pm$ 0.06	0.56 $\pm$ 0.03
	GALGUARD _p	0.64 $\pm$ 0.03	0.56 $\pm$ 0.02	0.67 $\pm$ 0.05	0.60 $\pm$ 0.06
	GALGUARD	0.67$\pm$0.04	0.62$\pm$0.04	0.68$\pm$0.04	0.64$\pm$0.05
PubMed	MetaAttack	0.89 $\pm$ 0.08	0.73 $\pm$ 0.04	0.85 $\pm$ 0.02	0.77 $\pm$ 0.03
	GALGUARD _p	0.90 $\pm$ 0.06	0.78 $\pm$ 0.03	0.85 $\pm$ 0.04	0.81 $\pm$ 0.04
	GALGUARD	0.90$\pm$0.01	0.87$\pm$0.02	0.89$\pm$0.02	0.89$\pm$0.01

in LLAGA creates a template-specific attack surface, while feature perturbation attacks can be highly effective across the evaluated graph-aware LLMs and can outperform structural attacks. A unified attack that combines feature and structural perturbations further demonstrates that robustness should be considered across both textual and relational inputs whenever graph-aware LLMs operate on text-attributed graphs.

To mitigate these vulnerabilities, we propose GALGUARD, an end-to-end defense that combines LLM-based feature correction with adapted GNN-based structural defenses. As graph-aware LLMs are considered for graph learning applications in domains such as social networks, healthcare, and finance, adversarial resilience should be treated as a core design requirement rather than an afterthought. We hope this study supports further work on robust graph-aware LLM architectures, broader evaluations of attack transferability across graph domains and LLM backbones, and practical defenses for deployment settings involving graph-structured data.

References

Achiam, J.; Adler, S.; Agarwal, S.; Ahmad, L.; Akkaya, I.; Aleman, F. L.; Almeida, D.; Altenschmidt, J.; Altman, S.; Anadkat, S.; et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*.

Ahmedt-Aristizabal, D.; Armin, M. A.; Denman, S.; Fookes, C.; and Petersson, L. 2021. Graph-based deep learning for medical diagnosis and analysis: past, present and future. *Sensors*, 21(14): 4758.

Anil, R.; Dai, A. M.; Firat, O.; Johnson, M.; Lepikhin, D.; Passos, A.; Shakeri, S.; Taropa, E.; Bailey, P.; Chen, Z.; et al. 2023. Palm 2 technical report. *arXiv preprint arXiv:2305.10403*.

Bi, B.; Liu, S.; Wang, Y.; Mei, L.; and Chen, X. 2024. Scalable Link Prediction on Large-Scale Heterogeneous Graphs with Large Language Models. *arXiv preprint arXiv:2401.13227*.

Boucher, N.; Shumailov, I.; Anderson, R.; and Papernot, N. 2022. Bad characters: Imperceptible nlp attacks. In *2022*

IEEE Symposium on Security and Privacy (SP), 1987–2004. IEEE.

Brannon, W.; Fulay, S.; Jiang, H.; Kang, W.; Roy, B.; Kabbara, J.; and Roy, D. 2023. Congrat: Self-supervised contrastive pretraining for joint graph and text embeddings. *arXiv preprint arXiv:2305.14321*.

Chai, Z.; Zhang, T.; Wu, L.; Han, K.; Hu, X.; Huang, X.; and Yang, Y. 2023. GraphLLM: Boosting Graph Reasoning Ability of Large Language Model. *arXiv preprint arXiv:2310.05845*.

Chandra, S.; Mishra, P.; Yannakoudakis, H.; Nimishakavi, M.; Saeidi, M.; and Shutova, E. 2020. Graph-based Modeling of Online Communities for Fake News Detection. *arXiv preprint arXiv:2008.06274*.

Chen, R.; Zhao, T.; Jaiswal, A.; Shah, N.; and Wang, Z. 2024. LLaGA: Large Language and Graph Assistant. In *Forty-first International Conference on Machine Learning*.

Chen, Z.; Mao, H.; Li, H.; Jin, W.; Wen, H.; Wei, X.; Wang, S.; Yin, D.; Fan, W.; Liu, H.; and Tang, J. 2023a. Exploring the Potential of Large Language Models (LLMs) in Learning on Graphs. *arXiv preprint arXiv:2307.03393*.

Chen, Z.; Mao, H.; Wen, H.; Han, H.; Jin, W.; Zhang, H.; Liu, H.; and Tang, J. 2023b. Label-free Node Classification on Graphs with Large Language Models (LLMs). *arXiv preprint arXiv:2310.04668*.

Chiang, W.-L.; Li, Z.; Lin, Z.; Sheng, Y.; Wu, Z.; Zhang, H.; Zheng, L.; Zhuang, S.; Zhuang, Y.; Gonzalez, J. E.; Stoica, I.; and Xing, E. P. 2023. Vicuna: An Open-Source Chatbot Impressing GPT-4 with 90%* ChatGPT Quality.

Conti, M.; Li, J.; Picek, S.; and Xu, J. 2022. Label-only membership inference attack against node-level graph neural networks. In *Proceedings of the 15th ACM Workshop on Artificial Intelligence and Security*, 1–12.

Dai, E.; Lin, M.; Zhang, X.; and Wang, S. 2023. Unnoticeable backdoor attacks on graph neural networks. In *Proceedings of the ACM Web Conference 2023*, 2263–2273.

Dai, H.; Li, H.; Tian, T.; Huang, X.; Wang, L.; Zhu, J.; and Song, L. 2018. Adversarial attack on graph structured data. In *International conference on machine learning*, 1115–1124. PMLR.

Dong, N.; Mücke, S.; and Khosla, M. 2022. Mucomid: A multitask graph convolutional learning framework for miRNA-disease association prediction. *IEEE/ACM Transactions on Computational Biology and Bioinformatics*, 19(6): 3081–3092.

Duan, K.; Liu, Q.; Chua, T.-S.; Yan, S.; Ooi, W. T.; Xie, Q.; and He, J. 2023. SimTeG: A Frustratingly Simple Approach Improves Textual Graph Learning. *arXiv preprint arXiv:2308.02565*.

Dubey, A.; Jauhri, A.; Pandey, A.; Kadian, A.; Al-Dahle, A.; Letman, A.; Mathur, A.; Schelten, A.; Yang, A.; Fan, A.; et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.

Duddu, V.; Boutet, A.; and Shejwalkar, V. 2020. Quantifying privacy leakage in graph embedding. In *MobiQuitous 2020-17th EAI International Conference on Mobile and Ubiquitous Systems: Computing, Networking and Services*, 76–85.

- Fan, W.; Ma, Y.; Li, Q.; He, Y.; Zhao, E.; Tang, J.; and Yin, D. 2019. Graph neural networks for social recommendation. In *The world wide web conference*, 417–426.
- Fatemi, B.; Halcrow, J.; and Perozzi, B. 2023. Talk like a Graph: Encoding Graphs for Large Language Models. *arXiv preprint arXiv:2310.04560*.
- Gong, N. Z.; and Liu, B. 2018. Attribute inference attacks in online social networks. *ACM Transactions on Privacy and Security (TOPS)*, 21(1): 1–30.
- Guo, J.; Du, L.; Liu, H.; Zhou, M.; He, X.; and Han, S. 2023. GPT4Graph: Can Large Language Models Understand Graph Structured Data ? An Empirical Evaluation and Benchmarking. *arXiv preprint arXiv:2305.15066*.
- Hamilton, W.; Ying, Z.; and Leskovec, J. 2017. Inductive representation learning on large graphs. *Advances in neural information processing systems*, 30.
- He, X.; Bresson, X.; Laurent, T.; Perold, A.; LeCun, Y.; and Hooi, B. 2024. Harnessing Explanations: LLM-to-LM Interpreter for Enhanced Text-Attributed Graph Representation Learning. In *The Twelfth International Conference on Learning Representations*.
- He, X.; Wen, R.; Wu, Y.; Backes, M.; Shen, Y.; and Zhang, Y. 2021. Node-level membership inference attacks against graph neural networks. *arXiv preprint arXiv:2102.05429*.
- Hu, W.; Fey, M.; Zitnik, M.; Dong, Y.; Ren, H.; Liu, B.; Catasta, M.; and Leskovec, J. 2020. Open graph benchmark: Datasets for machine learning on graphs. *Advances in neural information processing systems*, 33: 22118–22133.
- Huang, J.; Zhang, X.; Mei, Q.; and Ma, J. 2023a. Can LLMs Effectively Leverage Structural Information for Graph Learning: When and Why. *arXiv preprint arXiv:2309.16595*.
- Huang, X.; Han, K.; Bao, D.; Tao, Q.; Zhang, Z.; Yang, Y.; and Zhu, Q. 2023b. Prompt-based Node Feature Extractor for Few-shot Learning on Text-Attributed Graphs. *arXiv preprint arXiv:2309.02848*.
- Jin, W.; Li, Y.; Xu, H.; Wang, Y.; Ji, S.; Aggarwal, C.; and Tang, J. 2021. Adversarial attacks and defenses on graphs. *ACM SIGKDD Explorations Newsletter*, 22(2): 19–34.
- Jin, W.; Ma, Y.; Liu, X.; Tang, X.; Wang, S.; and Tang, J. 2020. Graph structure learning for robust graph neural networks. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, 66–74.
- Kipf, T. N.; and Welling, M. 2022. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations*.
- Kuang, H.; Xu, J.; Zhang, H.; Zhao, Z.; Zhang, Q.; Huang, X.; and Wei, Z. 2023. Unleashing the Power of Language Models in Text-Attributed Graph. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 8429–8441. Association for Computational Linguistics.
- Li, K.; Liu, Y.; Ao, X.; and He, Q. 2023. Revisiting graph adversarial attack and defense from a data distribution perspective. In *The Eleventh International Conference on Learning Representations*.
- Liu, C.; and Wu, B. 2023. Evaluating Large Language Models on Graphs: Performance Insights and Comparative Analysis. *arXiv preprint arXiv:2308.11224*.
- Liu, S.; Nie, W.; Wang, C.; Lu, J.; Qiao, Z.; Liu, L.; Tang, J.; Xiao, C.; and Anandkumar, A. 2023. Multi-modal molecule structure–text model for text-based retrieval and editing. *Nature Machine Intelligence*, 5(12): 1447–1457.
- Liu, Z.; He, X.; Tian, Y.; and Chawla, N. V. 2024. Can we soft prompt LLMs for graph learning tasks? In *Companion Proceedings of the ACM on Web Conference 2024*, 481–484.
- Ma, J.; Ding, S.; and Mei, Q. 2020. Towards more practical adversarial attacks on graph neural networks. *Advances in neural information processing systems*, 33: 4756–4766.
- Mavromatis, C.; Ioannidis, V. N.; Wang, S.; Zheng, D.; Adeshina, S.; Ma, J.; Zhao, H.; Faloutsos, C.; and Karypis, G. 2023. Train your own GNN teacher: Graph-aware distillation on textual graphs. *arXiv preprint arXiv:2304.10668*.
- Olatunji, I. E.; Hizber, A.; Sihlovec, O.; and Khosla, M. 2023a. Does black-box attribute inference attacks on graph neural networks constitute privacy risk? *arXiv preprint arXiv:2306.00578*.
- Olatunji, I. E.; Nejd, W.; and Khosla, M. 2021. Membership inference attack on graph neural networks. In *2021 Third IEEE International Conference on Trust, Privacy and Security in Intelligent Systems and Applications (TPS-ISA)*, 11–20. IEEE.
- Olatunji, I. E.; Rathee, M.; Funke, T.; and Khosla, M. 2023b. Private Graph Extraction via Feature Explanations. *Proceedings on Privacy Enhancing Technologies*.
- Pan, S.; Luo, L.; Wang, Y.; Chen, C.; Wang, J.; and Wu, X. 2024. Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering*.
- Peng, Y.; Lin, S.; Chen, Q.; Xu, L.; Ren, X.; Li, Y.; and Xu, J. 2024. ChatGraph: Chat with Your Graphs. *arXiv preprint arXiv:2401.12672*.
- Qin, Y.; Wang, X.; Zhang, Z.; and Zhu, W. 2023. Disentangled Representation Learning with Large Language Models for Text-Attributed Graphs. *arXiv preprint arXiv:2310.18152*.
- Shanthamallu, U. S.; Thiagarajan, J. J.; and Spanias, A. 2021. Uncertainty-matching graph neural networks to defend against poisoning attacks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, 9524–9532.
- Storn, R.; and Price, K. 1997. Differential evolution—a simple and efficient heuristic for global optimization over continuous spaces. *Journal of global optimization*, 11: 341–359.
- Su, B.; Du, D.; Yang, Z.; Zhou, Y.; Li, J.; Rao, A.; Sun, H.; Lu, Z.; and Wen, J.-R. 2022. A molecular multimodal foundation model associating molecule graphs with natural language. *arXiv preprint arXiv:2209.05481*.
- Sun, L.; Dou, Y.; Yang, C.; Zhang, K.; Wang, J.; Philip, S. Y.; He, L.; and Li, B. 2022. Adversarial attack and defense on graph data: A survey. *IEEE Transactions on Knowledge and Data Engineering*, 35(8): 7693–7711.

- Sun, S.; Ren, Y.; Ma, C.; and Zhang, X. 2023. Large Language Models as Topological Structure Enhancers for Text-Attributed Graphs. *arXiv preprint arXiv:2311.14324*.
- Tan, Y.; Zhou, Z.; Lv, H.; Liu, W.; and Yang, C. 2023. WalkLM: A Uniform Language Model Fine-tuning Framework for Attributed Graph Embedding. In *Thirty-seventh Conference on Neural Information Processing Systems*.
- Team, G.; Anil, R.; Borgeaud, S.; Wu, Y.; Alayrac, J.-B.; Yu, J.; Soricut, R.; Schalkwyk, J.; Dai, A. M.; Hauth, A.; et al. 2023. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*.
- Tian, Y.; Song, H.; Wang, Z.; Wang, H.; Hu, Z.; Wang, F.; Chawla, N. V.; and Xu, P. 2024. Graph neural prompting with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 19080–19088.
- Touvron, H.; Martin, L.; Stone, K.; Albert, P.; Almahairi, A.; Babaei, Y.; Bashlykov, N.; Batra, S.; Bhargava, P.; Bhosale, S.; et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.
- Veličković, P.; Cucurull, G.; Casanova, A.; Romero, A.; Liò, P.; and Bengio, Y. 2018. Graph Attention Networks. In *International Conference on Learning Representations*.
- Wang, J.; Zhang, S.; Xiao, Y.; and Song, R. 2021. A review on graph neural network methods in financial applications. *arXiv preprint arXiv:2111.15367*.
- Wang, X.; and Wang, W. H. 2022. Group property inference attacks against graph neural networks. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2871–2884.
- Wei, Y.; Fu, S.; Jiang, W.; Kwok, J. T.; and Zhang, Y. 2024a. Rendering graphs for graph reasoning in multimodal large language models. *arXiv preprint arXiv:2402.02130*.
- Wei, Y.; Huang, Q.; Kwok, J. T.; and Zhang, Y. 2024b. Kicgpt: Large language model with knowledge in context for knowledge graph completion. *arXiv preprint arXiv:2402.02389*.
- Wenkel, F.; Wolf, G.; and Knyazev, B. 2023. Pretrained Language Models to Solve Graph Tasks in Natural Language. In *ICML workshop on Structured Probabilistic Inference & Generative Modeling*.
- Wu, H.; Wang, C.; Tyshetskiy, Y.; Docherty, A.; Lu, K.; and Zhu, L. 2019. Adversarial examples on graph data: Deep insights into attack and defense. *arXiv preprint arXiv:1903.01610*.
- Wu, X.; Wu, H.; Zhou, X.; and Lu, K. 2021. CoG: a Two-View Co-training Framework for Defending Adversarial Attacks on Graph. *arXiv preprint arXiv:2109.05558*.
- Xie, H.; Zheng, D.; Ma, J.; Zhang, H.; Ioannidis, V. N.; Song, X.; Ping, Q.; Wang, S.; Yang, C.; Xu, Y.; et al. 2023. Graph-aware language model pre-training on a large graph corpus can help multiple graph applications. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 5270–5281.
- Yang, J.; Liu, Z.; Xiao, S.; Li, C.; Lian, D.; Agrawal, S.; Singh, A.; Sun, G.; and Xie, X. 2021. GraphFormers: GNN-nested Transformers for Representation Learning on Textual graph. In *Proceedings of the 35th Conference on Neural Information Processing Systems*.
- Yang, Z.; Cohen, W.; and Salakhudinov, R. 2016. Revisiting semi-supervised learning with graph embeddings. In *International conference on machine learning*, 40–48. PMLR.
- Ye, R.; Zhang, C.; Wang, R.; Xu, S.; and Zhang, Y. 2023. Natural Language is All a Graph Needs. *arXiv preprint arXiv:2308.07134*.
- Zhang, A.; and Ma, J. 2024. Defensevgae: Defending against adversarial attacks on graph data via a variational graph autoencoder. In *International Conference on Intelligent Computing*, 313–324. Springer.
- Zhang, J. 2023. Graph-toolformer: To empower llms with graph reasoning ability via prompt augmented by chatgpt. *arXiv preprint arXiv:2304.11116*.
- Zhang, X.; and Zitnik, M. 2020. Gnn-guard: Defending graph neural networks against adversarial attacks. *Advances in neural information processing systems*, 33: 9263–9275.
- Zhang, Z.; Chen, M.; Backes, M.; Shen, Y.; and Zhang, Y. 2022. Inference attacks against graph neural networks. In *31st USENIX Security Symposium (USENIX Security 22)*, 4543–4560.
- Zhang, Z.; Liu, Q.; Huang, Z.; Wang, H.; Lu, C.; Liu, C.; and Chen, E. 2021. Graphmi: Extracting private graph data from graph neural networks. *arXiv preprint arXiv:2106.02820*.
- Zhang, Z.; Wang, X.; Zhang, Z.; Li, H.; Qin, Y.; and Zhu, W. 2023. LLM4DyG: Can Large Language Models Solve Problems on Dynamic Graphs? *arXiv preprint arXiv:2310.17110*.
- Zhao, J.; Zhuo, L.; Shen, Y.; Qu, M.; Liu, K.; Bronstein, M.; Zhu, Z.; and Tang, J. 2023. GraphText: Graph Reasoning in Text Space. *arXiv preprint arXiv:2310.01089*.
- Zhu, Y.; Wang, Y.; Shi, H.; and Tang, S. 2024. Efficient tuning and inference for large language models on textual graphs. *arXiv preprint arXiv:2401.15569*.
- Zügner, D.; Akbarnejad, A.; and Günnemann, S. 2018. Adversarial attacks on neural networks for graph data. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*, 2847–2856.
- Zügner, D.; and Günnemann, S. 2019. Adversarial Attacks on Graph Neural Networks via Meta Learning. In *International Conference on Learning Representations (ICLR)*.

A Appendix

Organization: The Appendix provides additional context for the scope and generality of the study. Appendix A.1 gives a detailed taxonomy of graph-aware LLMs. Appendix A.2 reviews attacks against Graph Neural Networks (GNNs). Appendix A.3 summarizes NETTACK and METAATTACK, while Appendix A.4 details the node sequence injection attack algorithm for LLAGA. Appendix A.5 reports ablations on feature perturbation rates and base LLM choice. Appendix A.7 provides feature attack examples and the LLM corrector template, and Appendix A.8 discusses broader implications. **Code.** Our code will be released upon acceptance.

A.1 Graph Encoding-Based Taxonomy of LLM Adaptations

The integration of large language models (LLMs) with graph data now spans graph reasoning, representation, and inference, so a scope-aware taxonomy helps connect our experiments to the wider design space. We organize existing approaches by how they encode graphs for LLM consumption: *textual descriptions of graphs*, where graph information is rendered as natural language, and *learned graph projector*, where a trained projector maps graph data into embeddings or structured inputs that LLMs can process directly. This section reviews both families to situate the evaluated projector models within broader graph-aware LLM adaptations. Figure 5 illustrates the two approaches, and Table 7 summarizes their main differences.

Graph Representation as a Textual Description In the textual-description family, graph data is converted into natural language so that LLMs can process graph structures through prompts or instructions across different graph tasks. This direction includes work that converts graph data into textual formats (Guo et al. 2023; Liu and Wu 2023; Ye et al. 2023; Wenkel, Wolf, and Knyazev 2023; Wei et al. 2024b; Fatemi, Halcrow, and Perozzi 2023; Tan et al. 2023; Zhao et al. 2023; Wei et al. 2024a). For example, Tan et al. (2023) introduced a framework for attributed graph embedding that generates random-walk textual sequences and uses them to fine-tune LLMs. Similarly, Zhao et al. (2023) studied graph reasoning in text space by translating graph structures into natural language. Extending the same idea, Wei et al. (2024a) augmented textual graph representations with visual graph representations to support LLM reasoning over graph data. In multimodal settings, Liu et al. (2023) and Su et al. (2022) linked molecule graphs with textual descriptions for drug discovery and molecular retrieval. In addition, Huang et al. (2023a) examined how graph structure can be incorporated into LLM prompts and found that LLMs tend to process graph prompts as contextual paragraphs rather than explicit graph objects. This finding motivates careful prompt design in heterogeneous and large-scale graph tasks (Brannon et al. 2023; Zhang et al. 2023; Sun et al. 2023; Bi et al. 2024; Pan et al. 2024). Complementary work by (He et al. 2024) uses LLM-generated explanations as features to improve graph performance and interpretability.

A related line keeps the graph interface textual while integrating pretraining strategies with GNNs. For instance, Chen

et al. (2023a) proposed two LLM-GNN integration approaches: using LLMs to enhance node features for node classification and using LLMs as direct graph-task predictors. In a similar spirit, Chandra et al. (2020) used LLMs as text encoders and GNNs as graph structure encoders for fake news detection in online communities. Chen et al. (2023b) explored node classification without explicit labels by using LLMs to annotate nodes before training a GNN, and Mavromatis et al. (2023) used LLMs as feature extractors for GNNs.

For inference, Zhu et al. (2024) proposed a textual graph fine-tuning method that combines LLMs and GNNs through a tunable side structure alongside each LLM layer, improving training efficiency and inference speed. Duan et al. (2023) introduced SimTeG, which first fine-tunes an LLM on a downstream task such as node classification and then extracts node embeddings from the final hidden states to support textual graph learning. Additionally, Zhang (2023) proposed Graph-toolformer, which uses ChatGPT-supported prompt engineering to augment graph reasoning. Taken together, these approaches show that textual graph representations can support a range of graph-learning workflows, while also indicating that relational detail may be harder to preserve as graph size and structural complexity grow.

Graph Representation as a Learned Projector Learned-projector methods address limitations of textual descriptions by transforming graph data into embeddings or graph tokens that LLMs can process more directly. For instance, Chen et al. (2024) introduced LLAGA, a large language and graph assistant that keeps the general-purpose LLM interface while converting graph structures into graph tokens. Similarly, (Liu et al. 2024; Qin et al. 2023) incorporated graph structure into pre-trained LLMs through tailored disentangled graph neural network layers. Tian et al. (2024) proposed graph neural prompts for knowledge graphs, transforming graph information into embeddings that LLMs can use during inference. Huang et al. (2023b) introduced G-Prompt, a prompt-based node feature extractor for few-shot learning on text-attributed graphs that combines a learnable GNN adapter with task-specific prompts to capture neighborhood information.

Another line condenses graph information into fixed-length prefixes with graph transformers attached to LLM layers (Chai et al. 2023). Similarly, Yang et al. (2021) developed GraphFormers, which integrates text encoding and graph aggregation by nesting GNNs alongside transformer blocks. Peng et al. (2024) developed ChatGraph, an interactive interface for graph analysis through natural language, where graphs are represented as sequential paths and the LLM is fine-tuned on graph data.

We exclude projector adaptations that require retraining the LLM because they substantially increase computational cost without necessarily improving the graph-task performance relevant to our study. Instead, we focus on methods that use **frozen pre-trained LLMs** (Chen et al. 2024; Liu et al. 2024; Qin et al. 2023; Tian et al. 2024; Huang et al. 2023b). This setting reduces cost, supports integration with different LLM backbones, and still adapts the language model interface to graph tasks. We use two representative projector methods: **LLAGA** (Chen et al. 2024), which uses node-level templates

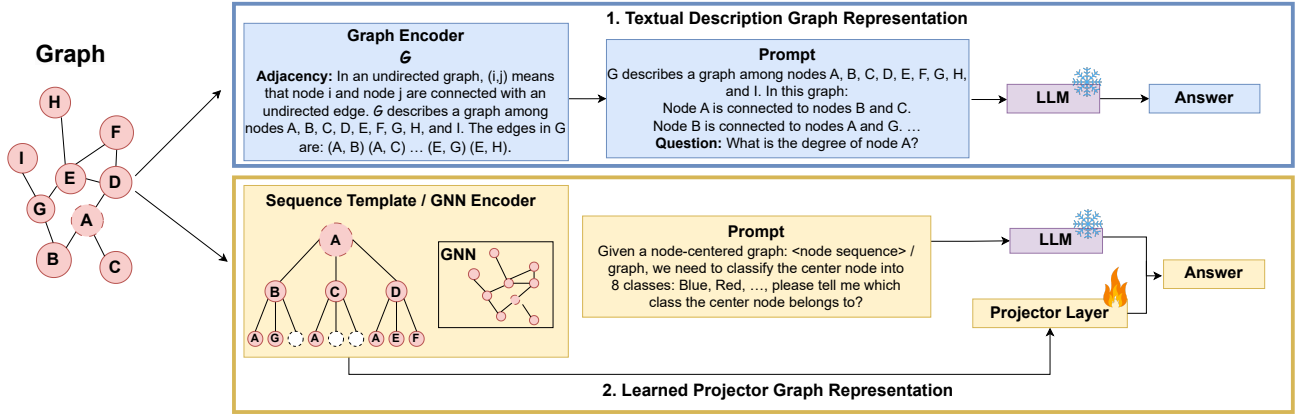


Figure 5: **Graph encoding-based adaptations of LLMs.** Textual-description methods express graph information in natural language (top), whereas learned-projector methods encode graphs with node templates that become sequences or with GNNs (bottom). In the frozen-projector cases shown here, the LLM is fixed and only the graph-to-token interface is trained.

Table 7: Comparison of graph encoding-based adaptations of LLMs.

Aspect	Textual Description Approach	Learned Projector Approach
Graph Encoding	Converts graph data (nodes, edges) into natural language descriptions.	Uses a learned projector to encode graph structures into embeddings or graph tokens.
LLM Modifications	No modification to the LLM.	No modification to the LLM.
Graph Structure Representation	Describes graph topology and relationships using textual narratives.	Encodes relational and structural information directly into vector representations.
Flexibility	Can only be used for simple graphs.	More flexible in representing complex graph structures.
Computational Cost	No additional training is required. Lower cost.	Lightweight projector is trained, typically a linear layer. Minimal cost.
Suitability for Graph Tasks	May lose some relational nuance due to limitations in textual descriptions.	Captures detailed structural information, improving performance on graph-specific tasks.

and a linear projector, and **GRAPHPROMPTER** (Liu et al. 2024), which uses a GNN-based projector to encode the graph.

This work. We do not evaluate textual-description methods because they can struggle to preserve detailed graph structure, especially as graph size increases. These methods are often most suitable for simpler graph queries, such as describing node degree or local relationships, while prompt token limits restrict detailed descriptions of larger graphs. From a robustness perspective, textual-description methods also expose a different attack surface: because the graph is represented as plain text (e.g., "Node A is connected to Node B"), they are naturally susceptible to conventional text-based adversarial attacks. Our study instead targets graph-specific vulnerabilities that arise when learned projectors encode structural and feature information for frozen LLMs.

A.2 Attacks on GNN

GNNs can be affected by several attack families: membership inference attacks (Olatunji, Nejd, and Khosla 2021; He et al. 2021; Conti et al. 2022), which determine whether a node was part of training; attribute inference attacks (Gong and Liu 2018; Olatunji et al. 2023a; Duddu, Boutet, and Shejwalkar 2020), which infer sensitive or missing attributes from outputs or embeddings; property inference attacks (Zhang et al. 2022; Wang and Wang 2022), which extract global graph properties; adversarial attacks (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019; Li et al. 2023; Dai et al. 2018; Sun et al. 2022), which degrade performance through subtle feature or structure perturbations; and graph reconstruction attacks (Zhang et al. 2021; Olatunji et al. 2023b), which reconstruct graph structure from learned representations or outputs.

Among these categories, adversarial attacks are directly relevant to model integrity because they are designed to reduce predictive performance. They introduce small, carefully crafted *perturbations* into node features or graph edges. These attacks are commonly divided into **poisoning attacks**, which manipulate training data, and **evasion attacks**, which modify inputs at *test time*. For example, Zügner, Akbarnejad, and Günnemann (2018) showed that perturbing graph structure or node features can substantially degrade GNN performance. Their method, **NETTACK**, uses greedy optimization to find minimal perturbations that maximize training loss, thereby identifying small feature or structure changes that disrupt predictions. Li et al. (2023) revisited graph adversarial attacks from a data distribution perspective and proposed a defense to mitigate distributional shifts caused by adversarial examples. Wu et al. (2019) analyzed adversarial examples on graph data and proposed defenses for detecting and counteracting adversarial modifications. **METAATTACK** (Zügner and Günnemann 2019) performs poisoning and evasion attacks by treating graph structure as an optimizable hyperparameter in a bi-level optimization problem. It uses meta-gradients to estimate how small structural perturbations affect attacker loss after training, enabling targeted graph updates that degrade performance.

In this paper, we use **NETTACK** (Zügner, Akbarnejad, and Günnemann 2018) and **METAATTACK** (Zügner and

Günnemann 2019) as representative adversarial attacks covering poisoning and evasion. Poisoning attacks target model integrity during training, while evasion attacks compromise performance at inference or test time.

A.3 How NETTACK and METAATTACK Work

NETTACK perturbs graph structure by adding and removing edges while preserving the degree distribution of the perturbed graph. It uses greedy optimization to find minimal perturbations that maximize the training loss of a surrogate model. The surrogate is a linearized 2-layer graph convolutional network (GCN) that approximates how structural changes affect classification. **NETTACK** aims to increase confidence in incorrect classifications by maximizing the distance between logits on the perturbed and clean graphs. It identifies candidate edge perturbations around a target node, evaluates a score for each candidate edge, selects the highest-scoring edge, and updates the graph by adding or removing that edge. Because **NETTACK** is targeted and causes misclassification of a single node, we follow (Zügner and Günnemann 2019) to convert it into an untargeted evaluation by randomly selecting test nodes, attacking them with **NETTACK** while considering all nodes in the network, and aggregating the results.

METAATTACK optimizes graph structure directly with gradient descent by treating the adjacency matrix as a hyperparameter. It formulates the attack as a bi-level optimization problem: the inner level minimizes training loss on the original graph, and the outer level minimizes loss on the poisoned graph. The method uses a linearized two-layer GCN surrogate to approximate how structural changes affect model performance. A score function estimates each candidate perturbation’s impact, and the attack flips the sign of meta-gradients for connected nodes to identify edges for removal. It then greedily adds or removes the highest-scoring edges to maximize attack impact.

For both methods, we use the unnoticeability constraint introduced by (Zügner, Akbarnejad, and Günnemann 2018), which limits changes to the graph degree distribution. Accordingly, only 10% of edges are perturbed.

In poisoning experiments with **LLAGA**, the perturbed graph structure from **NETTACK** or **METAATTACK** is used to generate node-level sequences, and the projector that maps these sequences to token embeddings is retrained. This setup preserves the **LLAGA** evaluation protocol while applying the same **NETTACK** and **METAATTACK** perturbation outputs used for the broader comparison. For **GRAPHPROMPTER**, the perturbed graph is re-encoded using a graph attention network (GAT), and the projector is retrained on the perturbed graph. In evasion experiments, the graph-aware LLM is trained on unperturbed data, and perturbations are introduced only at inference time.

A.4 Algorithm for Node Sequence Template Injection Attacks

Algorithm 1 describes the node sequence template injection attack for **LLAGA**. The model constructs a *fixed-shape* computational tree for each node u by sampling k neighbors per node up to depth d (lines 1–14). When a node has

Algorithm 1: Injection Attacks on Neighborhood Detail Template for Obtaining Node Sequence.

Input: Graph $G = (V, E)$, target node u , tree depth d , children size k
Output: Modified graph G' with injected nodes

```

1: procedure NEIGHBORHOODDETAILTEMPLATE( $u, d, k$ )
2:    $T \leftarrow \emptyset$  ▷ Initialize empty tree
3:    $T.\text{root} \leftarrow u$ 
4:   for level  $\ell = 1$  to  $d$  do
5:     for each node  $v$  at level  $\ell - 1$  do
6:        $N_v \leftarrow \text{Sample}(\mathcal{N}(v), k)$  ▷ Sample  $k$  neighbors
7:       if  $|N_v| < k$  then
8:         Fill remaining slots with placeholders
9:       end if
10:      Add  $N_v$  as children of  $v$  in  $T$ 
11:    end for
12:  end for
13:  return  $T$ 
14: end procedure

15: procedure ATTACK( $G, u, d, k, \text{AttackStrategy}$ )
16:    $G' \leftarrow G$  ▷ Create copy of original graph
17:    $T \leftarrow \text{NEIGHBORHOODDETAILTEMPLATE}(u, d, k)$ 
18:    $P \leftarrow \text{GetPlaceholders}(T)$ 
19:   if  $|P| > 0$  then
20:      $\mathcal{N}_2 \leftarrow 2\text{-hop neighborhood of } u$ 
21:     if  $\text{AttackStrategy} = \text{NI}$  then
22:        $V_{\text{non-adjacent}} \leftarrow V \setminus \mathcal{N}_2$ 
23:        $V_{\text{inject}} \leftarrow \text{Sample}(V_{\text{non-adjacent}}, |P|)$ 
24:       for each placeholder position  $p \in P$  do
25:          $v \leftarrow \text{Next node from } V_{\text{inject}}$ 
26:         Add edge  $(\text{parent}(p), v)$  to  $G'$ 
27:       end for
28:     else if  $\text{AttackStrategy} = \text{SI}$  then
29:        $V_{\text{non-adjacent}} \leftarrow V \setminus \mathcal{N}_2$ 
30:        $s \leftarrow \text{GetHighestDegreeNode}(V_{\text{non-adjacent}})$ 
31:       for first placeholder position  $p \in P$  do
32:         Add edge  $(\text{parent}(p), s)$  to  $G'$ 
33:       break ▷ Only use first placeholder
34:     end for
35:     else if  $\text{AttackStrategy} = \text{MSI}$  then
36:        $V_{\text{non-adjacent}} \leftarrow V \setminus \mathcal{N}_2$ 
37:        $S \leftarrow \text{GetNodesOrderedByDegree}(V_{\text{non-adjacent}})$ 
38:       for  $i = 1$  to  $|P|$  do
39:          $s \leftarrow \text{Get } i\text{-th node from } S$ 
40:          $p \leftarrow \text{Get } i\text{-th placeholder from } P$ 
41:         Add edge  $(\text{parent}(p), s)$  to  $G'$ 
42:       end for
43:     end if
44:   end if
45:   return  $G'$ 
46: end procedure

```

fewer than k neighbors, placeholders are inserted, which creates a template-specific attack surface. We exploit this mechanism by injecting malicious nodes into placeholder

positions through graph manipulation. The three strategies are *Non-Adjacent Injection (NI)*, which inserts nodes outside the 2-hop neighborhood of u (lines 21–27); *Supernode Injection (SI)*, which injects one high-degree node into a placeholder (lines 29–33); and *Multiple Supernode Injection (MSI)*, which distributes distinct high-degree nodes across placeholders (lines 35–43). These uninformative nodes corrupt the node sequence and can degrade projector training or inference.

A.5 Ablation Studies

This section provides the ablation results referenced in the main paper and clarifies which observed robustness trends persist across perturbation rates and LLM backbones.

Ablation Results: Varying Feature Perturbation Figure 6 reports performance for LLAGA and GRAPH-PROMPTER as the feature perturbation percentage varies. Increasing the proportion of perturbed features consistently reduces accuracy for both models, indicating that feature-level attack strength scales across the evaluated graph-aware LLM architectures rather than appearing only at a single perturbation budget.

Ablation Results: Does the Choice of LLM Affect Attack Performance? The results in Table 8 show that the base LLM has limited effect on clean performance and adversarial vulnerability within the LLAGA framework. Across Cora, Citeseer, and PubMed, Vicuna-7B and Llama2-7B variants achieve nearly identical clean accuracy and show similar degradation under poisoning and evasion attacks. For example, under METAATTACK evasion on Cora, performance is 0.44 ± 0.06 for Vicuna-7B and 0.43 ± 0.08 for Llama2-7B, which is within standard deviation. Similar patterns appear across datasets and attack types, suggesting that attack effectiveness is driven more by graph encoding than by the specific base LLM in this frozen-projector setting.

A.6 Results on the ArXiv Dataset

We also evaluate the attack on the larger ArXiv dataset, which has 169,343 nodes, 1,166,243 edges, and sparsity of 0.81. This experiment tests whether node sequence template attacks remain relevant beyond the smaller citation graphs in the main evaluation. As shown in Table 9, poisoning and evasion behave differently on ArXiv: poisoning is consistently more damaging than evasion across all three injection strategies, unlike the patterns observed on Cora, Citeseer, and PubMed. This suggests that dataset structure can change which attack stage is most harmful. Despite the low sparsity value, inserting even a single supernode can noticeably affect model performance.

A.7 Feature Attack Examples and Defense Template

Feature Attack Examples The examples in Appendix A.7 illustrate feature-level attacks using real Cora node features as concrete instances of a broader text-attributed graph vulnerability. The "input" corresponds to the text received by the LLM after perturbation. Although a human reader sees

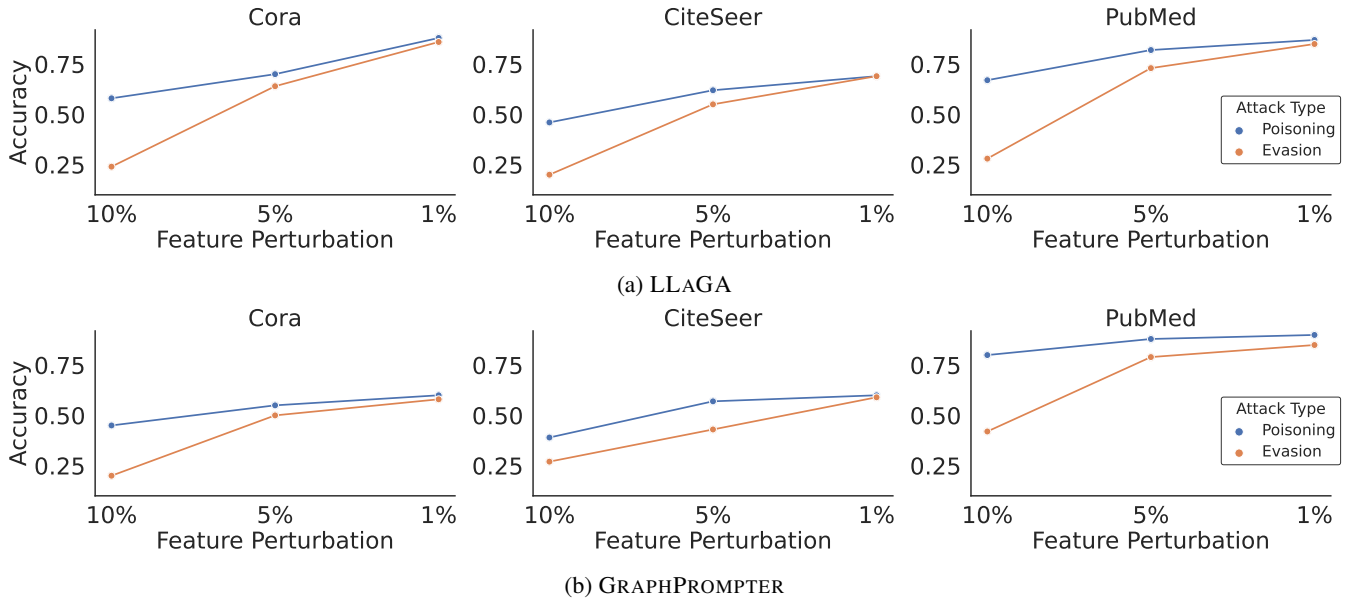


Figure 6: Performance of graph-aware LLMs as the percentage of perturbed textual features varies. Lower accuracy indicates a stronger attack effect.

Table 8: Impact of base model changes on LLAGA performance under clean and attacked settings. The results indicate that base model choice does not substantially change attack susceptibility within this graph-aware architecture.

Data	Base Model	Clean	Poisoning			Evasion		
			NETTACK	METAATTACK	MSI	NETTACK	METAATTACK	MSI
Cora	Vicuna-7B	0.89 ± 0.07	0.87 ± 0.04	0.79 ± 0.03	0.82 ± 0.04	0.55 ± 0.09	0.44 ± 0.06	0.30 ± 0.05
	Llama2-7B	0.90 ± 0.03	0.86 ± 0.02	0.77 ± 0.05	0.82 ± 0.02	0.56 ± 0.07	0.43 ± 0.08	0.31 ± 0.01
Citeseer	Vicuna-7B	0.71 ± 0.04	0.64 ± 0.05	0.63 ± 0.04	0.62 ± 0.02	0.59 ± 0.04	0.50 ± 0.03	0.24 ± 0.05
	Llama2-7B	0.71 ± 0.05	0.65 ± 0.03	0.60 ± 0.03	0.61 ± 0.04	0.58 ± 0.03	0.48 ± 0.05	0.26 ± 0.03
PubMed	Vicuna-7B	0.90 ± 0.04	0.89 ± 0.03	0.89 ± 0.08	0.81 ± 0.03	0.84 ± 0.05	0.73 ± 0.04	0.76 ± 0.03
	Llama2-7B	0.91 ± 0.06	0.88 ± 0.04	0.90 ± 0.02	0.80 ± 0.04	0.85 ± 0.03	0.75 ± 0.02	0.77 ± 0.02

Table 9: Node sequence template injection results across datasets and injection strategies, including the larger ArXiv setting. *NI*, *SI*, and *MSI* denote non-adjacent, single-supernode, and multiple-supernode injection, respectively. Clean reports unattacked model performance.

Data	Attack	NI	SI	MSI
Cora	Clean	—	0.89 ± 0.07	—
	Poisoning	0.86 ± 0.03	0.84 ± 0.06	0.82 ± 0.04
	Evasion	0.42 ± 0.08	0.36 ± 0.05	0.30 ± 0.05
Citeseer	Clean	—	0.71 ± 0.04	—
	Poisoning	0.69 ± 0.05	0.66 ± 0.03	0.62 ± 0.02
	Evasion	0.36 ± 0.06	0.30 ± 0.04	0.24 ± 0.05
PubMed	Clean	—	0.90 ± 0.04	—
	Poisoning	0.88 ± 0.04	0.85 ± 0.05	0.81 ± 0.03
	Evasion	0.85 ± 0.07	0.82 ± 0.04	0.76 ± 0.03
ArXiv	Clean	—	0.73 ± 0.02	—
	Poisoning	0.50 ± 0.03	0.43 ± 0.04	0.42 ± 0.03
	Evasion	0.71 ± 0.06	0.69 ± 0.02	0.60 ± 0.03

the original and perturbed text as identical or nearly identical, the LLM processes different Unicode characters or a different character order, which can change model behavior. Table 10 shows a homoglyph example, and Table 11 shows a reordering example.

Feature Corrector Defense Template We design a feature corrector module to mitigate imperceptible attacks, including homoglyph substitutions and character reorderings. The module is implemented as a prompt-based LLM defense. The prompt asks the LLM to act as a linguistic validator that detects and corrects subtle perturbations that may be invisible to humans.

The prompt template is shown below. It states the task and output format explicitly, which improves the reliability of the correction step.

A.8 Detailed Discussion

Our findings provide several insights into the robustness and design tradeoffs of graph-aware LLMs. As graph-structured

Prompt for Feature Corrector

You are a linguistic validator. Your task is to detect and correct any subtle, imperceptible text perturbations such as homoglyph substitutions (e.g., replacing "o" with "0") and character reorderings. These changes are often used to attack machine learning models. Given a text string, carefully analyze it. If you identify any perturbations, provide the corrected, original-looking version of the text. If no perturbations are found, return the original text unchanged.

Example 1 (Homoglyph Substitution):

Input: Title: The megaprior heuristic fr discvring prtin squnc pttrens Abstract: Svrl cmputr lgrithms fr discvring pttrens in grups f prtin squncs r in us tht r bsd on fitting th prmtrs f sttisticl mdl to grup of rlted squncs.

Output: Title: The megaprior heuristic for discovering protein sequence patterns Abstract: Several computer algorithms for discovering patterns in groups of protein sequences are in use that are based on fitting the parameters of a statistical model to a group of related sequences.

Example 2 (Character Reordering):

Input: Title: Appliatcions of machine lreanign: a medical foall-up study Abstract: This ppaer dscribes pmerlinary work that aims to aplpy smoe learning sttraegies to a meidcal flolow-up study. An inestivagiottn of the apclpiation of three machine larening algorithms-lR, FOIL and InductH to identify risk frcatos that govern the clopsuspension cure rtae has been mdae.

Output: Title: Applications of machine learning: a medical follow up study Abstract: This paper describes preliminary work that aims to apply some learning strategies to a medical follow-up study. An investigation of the application of three machine learning algorithms-lR, FOIL and InductH to identify risk factors that govern the colposuspension cure rate has been made.

Example 3 (Mixed Homoglyph & Reordering):

Input: Title: Submitted to NIPS96, Sctin: Applications. Prfrnc: Oral prsentatin Reinforcement Lrning for Dynmic Channl Allocation in Abstract: In ellulr tlphne systms, n imrtnt prblm is to dynmiclly alloct th cmmunictin rsurc (chnnls) so as to mximize srvc in stchstic cllr nvirnmt. This prblem is naturally frmultd s dynmic prgmming prblm and we us a reinforcement lerning (RL) mthd to find dynmic chnnl llection plicies that ar bttr than prvius heuristic slutins.

Output: Title: Submitted to NIPS96, Section: Applications. Preference: Oral presentation Reinforcement Learning for Dynamic Channel Allocation in Abstract: In cellular telephone systems, an important problem is to dynamically allocate the communication resource (channels) so as to maximize service in a stochastic caller environment. This problem is naturally formulated as a dynamic programming problem and we use a reinforcement learning (RL) method to find dynamic channel allocation policies that are better than previous heuristic solutions.

Your Task: Text to correct: {perturbed_text_here}

Corrected Text:

Figure 7: Prompt template for the LLM-based feature corrector module. The prompt defines the role, task, and expected output format for correction.

Table 10: Illustration of the imperceptible homoglyph attack on textual node features. The LLM processes the perturbed text as distinct characters, while the rendered text appears nearly identical to a human reader.

Original Feature	What LLM Sees	What a Human Sees
Title: The megaprior heuristic for discovering protein sequence patterns Abstract: Several computer algorithms for discovering patterns in groups of protein sequences are in use that are based on fitting the parameters of a statistical model to a group of related sequences.	Title: The mgpprir huristic fr discvring prtin squnc pptrns Abstract: Svrl cmputr lgrithms fr discvring pptrns in grups f prtin squncs r in us tht r bsd on fitting th prmtrs f sttisticl mdl to grup of rltd squncs.	Title: The megaprior heuristic for discovering protein sequence patterns. Abstract: Several computer algorithms for discovering patterns in groups of protein sequences are in use that are based on fitting the parameters of a statistical model to a group of related sequences.

Table 11: Illustration of the imperceptible reordering attack on textual node features. The LLM receives a different character order, while the rendered text remains nearly identical to a human reader.

Original Feature	What LLM Sees	What a Human Sees
Title: Applications of machine learning: a medical follow up study. Abstract: This paper describes preliminary work that aims to apply some learning strategies to a medical follow-up study. An investigation of the application of three machine learning algorithms-1R, FOIL and InductH to identify risk factors that govern the colposuspension cure rate has been made.	Title: Appliatcions of machine lreanign: a medical foall-up study Abstract: This ppaer dsceribes pmerlinary work that aims to aplpy smoe learning sttraegies to a meidcal flolow-up study. An inestivagiotn of the aplcpiation of three machine larening algorithms-1R, FOIL and InductH to identfy risk frcatos that govern the clopsuspension cure rtae has been mdae.	Title: Applications of machine learning: a medical follow up study Abstract: This paper describes preliminary work that aims to apply some learning strategies to a medical follow-up study. An investigation of the application of three machine learning algorithms-1R, FOIL and InductH to identify risk factors that govern the colposuspension cure rate has been made.

data is increasingly integrated into LLM pipelines, evaluation should cover not only predictive performance but also vulnerability to adversarial manipulation. Our study of two representative graph-aware LLMs, LLAGA and GRAPHPROMPTER, shows that poisoning and evasion attacks adapted from the GNN setting can expose important weaknesses. The main observations are discussed below.

Impact of Graph Encoding on Adversarial Vulnerability

A central observation is that the graph integration mechanism can shape adversarial susceptibility. The node sequence template in LLAGA creates an exploitable vulnerability that can substantially degrade performance. This attack surface highlights a design tradeoff for graph-aware LLMs: node-level templates can incorporate graph structure effectively, but they may introduce attack vectors that do not arise in traditional GNNs. In contrast, GRAPHPROMPTER uses a GNN-based

projector and is comparatively more resilient in our experiments, suggesting that hybrid graph-language architectures can offer robustness advantages under some attack conditions.

Sensitivity of Graph-aware LLMs to Feature Perturbations

Our results also show that feature perturbation attacks are effective across both graph-aware LLMs. Although LLAGA and GRAPHPROMPTER encode graph structure differently, both are sensitive to imperceptible textual changes in node features. This reflects a broader issue for LLM-based graph systems: strong text-processing capability does not automatically imply robustness to adversarial text perturbations. Subtle node-attribute changes can degrade model performance without being visible to humans, which is especially concerning in security-sensitive applications. Text sanitization methods such as those discussed by (Boucher

et al. 2022) are relevant but incomplete; using an LLM as a corrector offers one practical mitigation by detecting and repairing suspicious inputs before downstream graph processing.

Data-dependent Variability of Poisoning and Evasion Attacks on Graph-aware LLMs Attack effectiveness is not uniform across datasets. On Cora, Citeseer, and PubMed, attacks often produce substantial degradation, while ArXiv shows a different relationship between attack strategy and graph structure. This variability indicates that dataset properties such as size, sparsity, and local connectivity can influence graph-aware LLM robustness. Evaluations should therefore consider both algorithmic design and the characteristics of the target domain or data distribution before generalizing robustness claims.

Implications for Graph-aware LLM Development These vulnerabilities have broader implications for graph-aware LLM development and deployment. As these systems are adopted for tasks previously dominated by GNNs, model design should account for both task performance and adversarial exposure. More robust graph encoding techniques and defenses are needed, especially for feature perturbations that exploit the textual interface. Understanding how attacks transfer from GNNs to graph-aware LLMs can guide future defenses for models operating on graph-structured data across application domains.

Difficulty in Applying Existing GNN defenses Against Adversarial Attacks on Graph-aware LLMs Defenses on adversarial attacks based on feature perturbation. Adversarial attacks on GNNs have mainly emphasized structural perturbations because feature-based perturbations often fail to change the predicted class of a target node (Dai et al. 2018; Wu et al. 2019; Zügner and Günnemann 2019). As a result, there are *no known defenses against feature-based attacks on GNNs* for this setting. In graph-aware LLMs, however, feature perturbations can be effective because textual node attributes are processed directly by the LLM component. This creates a defense requirement that covers both graph structure and text features across the full graph-to-language pipeline. OCR-based preprocessing and other text sanitization techniques may help, but imperceptible attacks such as homoglyph substitutions remain difficult because OCR can misinterpret homoglyphs more often than unperturbed text (Boucher et al. 2022). This motivates our LLM-based self-correction mechanism for feature perturbations.

Defenses on adversarial attacks based on structural perturbation. Because structure-based attacks transfer from GNNs to graph-aware LLMs, especially for evasion, it is tempting to assume that GNN defenses will also transfer. Our setting shows that this assumption is incomplete: *graph-aware LLMs are also susceptible to feature perturbations, rendering these defenses ineffective* when the defenses rely on clean node features.

One GNN defense strategy makes the adjacency matrix

trainable so edge weights can adapt during training. This approach assigns lower weights to edges between dissimilar nodes and higher weights to edges between similar nodes. Several **preprocessing-based** defenses build on this idea, including removing low-similarity edges (Wu et al. 2019) and randomized edge dropping (Dai et al. 2018). These methods depend on clean node features to compute reliable similarity scores. When node features are perturbed, these defenses can fail and may even introduce incorrect edges. We illustrate this issue for a random node in Cora (Figure 8). The clean feature similarity graph preserves some original edges while adding some incorrect ones, whereas the graph reconstructed from perturbed features preserves no original connections and produces an entirely incorrect reconstruction.

Another defense line reduces dependence on graph structure. For example, (Shanthamallu, Thiagarajan, and Spanias 2021) trains a surrogate predictor that depends only on node features and uses uncertainty matching to extract graph information from the GNN. Similarly, Wu et al. (2021) trains sub-models on feature and structural views and distills knowledge using confident unlabeled data. These methods also assume unperturbed node features, which does not hold when both structure and features are adversarially perturbed. Thus, structural defenses developed for GNNs are not directly sufficient for graph-aware LLMs, motivating defenses tailored to the combined text-and-graph setting.

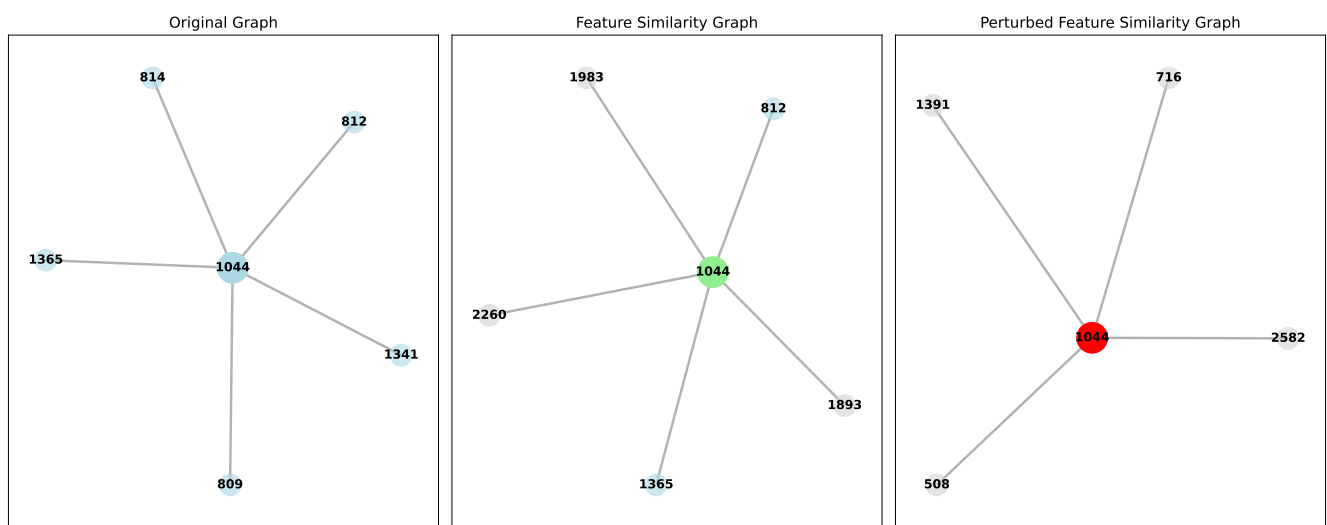


Figure 8: Edges reconstructed for random node 1044, illustrating how feature-dependent structural defenses can change under textual perturbation. **Original (left)** shows ground-truth edges from the original graph. **Feature Similarity (middle)** shows edges reconstructed from clean node features, with blue nodes indicating preserved original connections. **Perturbed Feature Similarity (right)** shows edges reconstructed from perturbed node features.